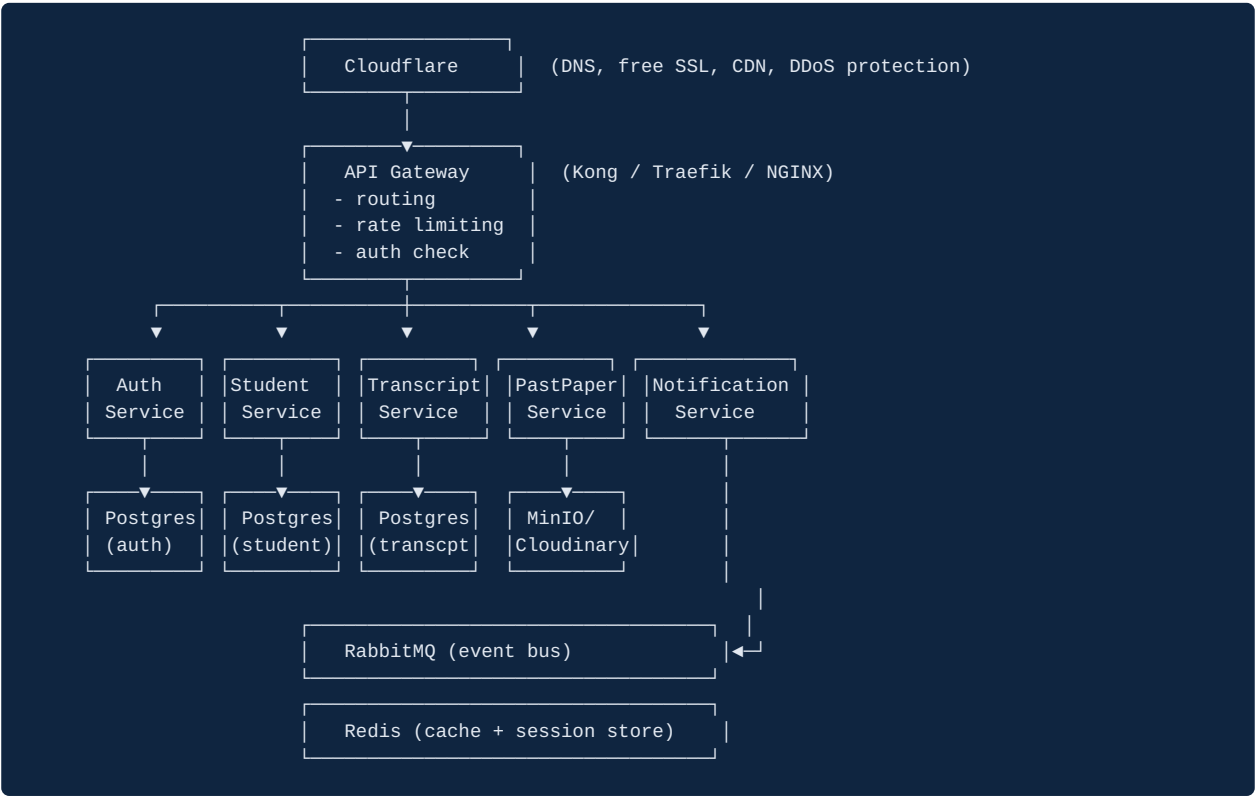


Microservice Scalable Institution Student Portal — Project Plan

Goal: SaaS student portal, microservice architecture, must sustain 1,000 concurrent students without crashing (3,000 registered/total base). Sections (Dashboard, Transcript, Past Papers, etc.) added incrementally, each proven operational before moving to the next.

1. Architecture Overview



Why this shape: each "portion" you mentioned (Dashboard, Transcript, Past Papers) maps naturally to its own service. The Dashboard itself isn't really a service that owns data — it's an **aggregator/BFF (Backend-For-Frontend)** that calls the other services and composes the view. This lets you build and prove one service at a time, exactly as you planned.

2. Services — what each one should do

Service	Responsibility	Notes
API Gateway	Single entry point, routes requests, rate-limits, terminates auth tokens	Kong or Traefik (both free/open source)
Auth Service	Signup/login, JWT issuing & refresh, role management (student/admin/lecturer), password reset	Own Postgres DB. This is priority #1 to build — everything else depends on it.
Student Service	Student profile, enrollment status, program/department info	Own Postgres DB
Dashboard/BFF Service	Aggregates data from other services into one payload for the frontend home screen	No DB of its own (or a thin cache)
Transcript Service	Grades, GPA calc, transcript generation (PDF)	

Service	Responsibility	Notes
		Own Postgres DB + PDFKit or WeasyPrint for PDF export (you already have WeasyPrint experience from the Christa's project)
Past Papers Service	Upload/store/search past exam papers by course/year/semester, download tracking	File storage (see §3) + Postgres for metadata (course, year, uploader, tags)
Notification Service	Email/SMS/push notifications (e.g., new past paper uploaded, grade posted)	Listens to RabbitMQ events, doesn't need its own frontend-facing API
Admin/Institution Service (later)	Multi-tenant management if you support more than one institution	Needed only once you go beyond a single pilot school

Event-driven pattern: when Transcript Service posts a new grade, it publishes an event to RabbitMQ; Notification Service picks it up and emails the student. This keeps services decoupled — one being slow/down doesn't take others down with it.

3. Image/File Storage: Cloudinary vs MinIO — my recommendation

You mentioned "Milo" — I'm assuming you meant **MinIO** (self-hosted, S3-compatible object storage), since that's the natural comparison to Cloudinary. Here's the honest breakdown:

	Cloudinary	MinIO
Cost	Free tier: 25 credits/mo (~25GB storage+bandwidth combined)	Free forever, but you pay in server disk/RAM to host it
Best for	Profile photos, banners — anything needing resizing/cropping/transformations on the fly	Bulk documents — past-paper PDFs, transcripts
CDN	Built-in, global	You'd need to add Cloudflare in front of it yourself
Setup effort	Near zero — just an SDK call	You manage a container, backups, access policies
Scaling to 1000 concurrent users	Handled by Cloudinary's infra, not yours	Handled by your own server — more load on your cluster

My recommendation — use both, split by purpose: - **Cloudinary** for user-facing images (profile pictures, institution logos, banners) — free tier is plenty for 1,000–3,000 students, and image transformation/CDN is genuinely useful for a content-creator-minded person like you who cares about visual polish. - **MinIO** for Past Papers (PDFs/docs) — these are large, numerous, and don't need image transformations, so Cloudinary's free tier would burn through quota fast. MinIO is free and you already have it in your toolkit from FlowDesk, so there's no new learning curve.

This split also happens to be a nice thing to point out to examiners: you made a *deliberate* storage decision per data type rather than using one tool for everything.

4. Database Strategy (PostgreSQL)

For a solo final-year build under time pressure, don't over-engineer "one fully isolated DB per service" on day one. Two realistic options:

- **Option A (recommended for your timeline):** One PostgreSQL instance, **separate schema per service** (`auth.*`, `student.*`, `transcript.*`, `pastpapers.*`). Each microservice connects only to its own schema/credentials. This keeps true service boundaries in your code while being much easier to deploy, back up, and pool connections against — critical when you're trying to prove 1,000 concurrent connections without your DB falling over.

- **Option B (textbook-pure):** Fully separate DB per service, like your FlowDesk plan. More "correct" microservices theory, but multiplies your connection-pooling and ops overhead right when you need to focus on load performance. Only go this route if you have time to spare after Option A is proven at scale.

Either way: use **PgBouncer** (free) in front of Postgres for connection pooling — this is the single highest-leverage thing you can do to survive 1,000 simultaneous students, since Postgres itself chokes on too many raw open connections.

5. How you actually prove "1,000 concurrent, 3,000 total"

This is a claim you need to *demonstrate*, not just architect for. Plan:

1. **Load testing tool:** k6 (free, scriptable, made for exactly this) or Locust (Python-based, also free).
2. **Horizontal scaling:** run multiple replicas of each service behind the gateway. Kubernetes (k3s — lightweight, free) handles this with `replicas: N` and auto-restarts crashed pods.
3. **Caching:** Redis in front of read-heavy endpoints (e.g., dashboard aggregation, past-paper listings) so repeated requests don't hit Postgres every time.
4. **Where to run the load test:**
5. **Oracle Cloud Always Free tier** gives you 4 ARM CPUs + 24GB RAM, genuinely free forever — enough to run a small k3s cluster and realistically simulate 1,000 concurrent virtual users. This is the single best "free money" resource for your defense demo.
6. Alternatively, DigitalOcean/Linode droplets (~\$20–40/month) if you want x86 simplicity and don't mind the small cost.
7. **Deliverable for your defense:** a k6 report/graph showing response times and error rates at 1,000 and 3,000 simulated users — this is concrete, gradeable evidence, not just a claim in your report.

6. Full Software List

Category	Tool	Cost
Backend framework	Node.js/Express or Spring Boot (pick whichever you're fastest in — you've used both)	Free
API Gateway	Kong or Traefik	Free
Database	PostgreSQL + PgBouncer	Free
Cache	Redis	Free
Message broker	RabbitMQ	Free
Containers	Docker	Free
Orchestration	k3s (lightweight Kubernetes)	Free
CI/CD	GitHub Actions	Free (2,000 min/mo on free tier)
Monitoring	Prometheus + Grafana	Free
Load testing	k6	Free
Image storage/CDN	Cloudinary	Free tier (25 credits/mo)
Document storage	MinIO	Free (self-hosted)
PDF generation	WeasyPrint (you already know this) or PDFKit	Free
Frontend	React (or your preference)	Free
DNS/CDN/SSL	Cloudflare	Free
Hosting	Oracle Cloud Always Free tier	Free
Domain name	Namecheap/Google Domains	~\$10–15/year

Category	Tool	Cost
Version control	GitHub	Free
API docs	Swagger/OpenAPI	Free

Total realistic out-of-pocket cost: just the domain name (~\$10–15/year). Everything else has a free tier sufficient for a final-year project + defense demo. If you later commercialize this as a real SaaS post-graduation, you'd budget for paid hosting tiers, but not for now.

7. SaaS / Multi-Tenancy Note

Since you called this a SaaS platform, decide now (even if you only launch with one pilot institution): add an `institution_id` column across your core tables from day one. It costs almost nothing to add now and saves a painful migration later if a second school wants to use the portal. You don't need full schema-per-tenant complexity yet — just make sure every query is scoped by `institution_id`.

8. Timeline: August 10 → End of September

Given the tight window, priority order is: **prove the scalable core first, then layer features on top of it** — exactly matching your instinct to add sections one at a time.

Week 1 (Aug 10–16): Foundations

- Finalize architecture diagram + DB schema (Option A schema-per-service)
- Set up GitHub repo, project board (Trello, like FlowDesk)
- Provision Oracle Cloud Always Free instance, install Docker + k3s
- Set up Postgres + PgBouncer + Redis + RabbitMQ containers
- Set up CI/CD skeleton (GitHub Actions: build + test on push)

Week 2 (Aug 17–23): Auth Service + Gateway

- Build Auth Service (signup/login/JWT/roles) — this unblocks everything else
- Deploy API Gateway, route `/auth/*` through it
- Write first k6 test script, baseline load test Auth Service alone

Week 3 (Aug 24–30): Student Service + Dashboard

- Build Student Service (profile, enrollment)
- Build Dashboard/BFF service aggregating Auth + Student data
- Frontend: login flow + basic dashboard shell
- Load test Auth + Student together at 100 → 500 concurrent users

Week 4 (Aug 31–Sep 6): Transcript Service

- Build Transcript Service + PDF export (WeasyPrint)
- Wire into Dashboard
- Frontend: transcript view/download
- Load test at 1,000 concurrent — identify and fix bottlenecks (this is usually where you discover you need more caching or connection pooling)

Week 5 (Sep 7–13): Past Papers Service

- Build Past Papers Service + MinIO integration
- Upload/search/download by course, year, semester
- Frontend: Past Papers section
- Notification Service: "new past paper uploaded" events via RabbitMQ

Week 6 (Sep 14–20): Full-system scale test

- Run the full stack together at 1,000 and then 3,000 simulated users
- Fix whatever breaks (this is normal — expect at least one real bottleneck here)
- Add Prometheus/Grafana dashboards showing live system health during the test

Week 7 (Sep 21–27): Polish & Documentation

- UI polish across all sections
- Write up architecture documentation, diagrams, load-test results for your report
- Record a demo video as backup in case live demo has issues on defense day

Week 8 (Sep 28–30+): Buffer

- Buffer for anything that slipped
- Rehearse defense walkthrough
- Final deployment check

9. Key Risks to Watch

- **Postgres connection exhaustion** is the #1 thing that will "crash" your system at 1,000 users if you skip PgBouncer — don't skip it.
- **Building too many services before load-testing any of them** — test Auth alone before adding Student, don't wait until everything is built to discover a bottleneck.
- **Oracle Cloud Always Free capacity limits** — 4 ARM CPUs/24GB RAM is generous but not infinite; if your load test itself needs to run from a separate machine (k6 running locally or on a second free-tier instance is fine — don't run the load generator on the same box as the system under test, or you'll get skewed results).

If you'd like, I can generate the actual PlantUML/architecture diagrams, the initial Postgres schema, or scaffold the Auth Service code next — just say which piece you want to start with.